```
#start
pushq %rbp              # save old %rbp
 movq %rsp, %rbp        # %rbp = new frame pointer
subq $32, %rsp          # allocate stack
movq $7, %rax           # %rax=7
movq %rax, -8(%rbp)     # roll saved in stack
                        -> Flags : unchanged
```

```
#Do-While Loop :L1
movq $0, %rsi           # %rsi = 0
movq %rsi, -16(%rbp)    # total = 0 on stack
                        -> Flags: ZF=1 (after mov $0)
```

```
#Printf
movq $msg_how_many, %rdi
xorq %rax, %rax   #%rax = 0
call printf
                        -> Flags:undefined after call (caller-saved)
```

```
#scanf numOfDice
movq $scan_int, %rdi
leaq -24(%rbp), %rsi
xorq %rax, %rax
call scanf
movq -24(%rbp), %rdi   #%rdi = numOfDice
                        -> Flags: may be set by scanf return value in %rax
```

```
#validation : numOfDice<=0?
cmpq $0, %rdi     #Sets ZF=1 if ,%rdi==0
jle L3            #Jump if ZF=1 or or SF≠OF
if invalid: print error -> jmp L1 (loop back)
```

```
#for loop init: L4
movq $0, %rcx           # %rcx = i = 0
movq -8(%rbp), %rax     # %rax = Roll
movq -24(%rbp), %rdi    #%rdi = numOfDice
```

```
#Loop condition: L5
cmpq %rdi, %rcx   # Compares i and numOfDice-> sets
OF/CF/SF/ZF
 jl L6                  # Jump if i < numOfDice (SF≠OF)
```

```
#Loop body: L6
movq $0x41C64E6D, %r9   #%r9 = multiplier
imulq %r9, %rax         # %rax *= multiplier-> sets
OF/CF/SF/ZF
addq %rdi, %rax         #%rax += numOfDice -> sets
OF/CF/SF/ZF
movq $6, %r8            #6 is divisor
cqto                    # Sign-extend %rax ->
%rdx:%rax
idivq %r8               # Divide -> quotient in %rax,
remainder in %rdx-> Sets ZF/SF/OF (CF undefined)
movq %rdx, %rax         #movq %rdx, %rax
addq $1, %rax           #%rax += 1 (final die value)->
Sets OF/CF/SF/ZF
```

```
#printf "Die %d: %ld\n"
pushq %rax, %rcx, %rdi  # Preserve Roll, i, numOfDice
movq %rax, %rdx         #%rdx = Roll (3rd arg)
movq %rcx, %rsi         # %rsi = i
addq $1, %rsi           #%rsi = i+1 (2nd arg)
movq $msg_die, %rdi     #%rdi = format (1st arg)
xorq %rax, %rax         -> %rax = 0
call printf
popq %rdi, %rcx, %rax   # Restore preserved values
```

```
#Update total
movq -16(%rbp), %rsi    # %rsi = total
addq %rax, %rsi         # %rsi += Roll -> sets OF/CF/SF/ZF
movq %rsi, -16(%rbp)    # total updated on stack
addq $1, %rcx           # i++
```

Repeat until i >= numOfDice

```
#After loop
movq %rax, -8(%rbp)     # Save last Roll as seed
[printf total sum]      #Similar to above: %rsi=total,
%rdi=format, %rax=0
[printf "Roll again?"]  -> %rdi=msg_again, %rax=0
```

```
#[scanf choice]
leaq -25(%rbp), %rsi  # %rsi = address of choice
movq $scan_char, %rdi
xorq %rax, %rax
  call scanf
```

```
movzbl -25(%rbp), %ebx  #%ebx = choice (zero-extend
byte)
cmpb $'y', %bl          # Compare -> sets ZF/SF
je L1                   #  Jump if ZF=1
cmpb $'Y', %bl          #Compare -> sets ZF/SF
je L1                   # Jump if ZF=1
```

```
#endd
addq $32, %rsp
popq %rbp
ret
Returns to caller (%rax usually 0 or garbage)
```